Module 13 Assignment Submission

# Project Reflection: Testimonial Collection SaaS

**Course:** Agentic Engineering Workflow • **GitHub:** https://github.com/Enjamam/Testimonials

## 1. What worked well with Claude Code & the Agentic Workflow?

Applying the agentic pair-programming workflow with structured context files (`CLAUDE.md`, `CONTEXT.md`) provided exceptional development velocity and architectural consistency. Explicit boundaries prevented scope creep and kept business logic strictly on the server. The plan → implement → verify rhythm worked seamlessly: the agent generated database migrations, Cashier Stripe billing, and Inertia React 19 interfaces while adhering strictly to tenant isolation policies. Integrating Model Context Protocol (MCP) tooling—specifically Chrome DevTools MCP for live DOM and viewport screenshot audits—proved game-changing for validating responsive layouts and celebratory micro-interactions without guesswork.

## 2. Where did it go wrong, and how was it fixed?

The primary challenge arose during profile photo uploads and settings view refinement. Initially, settings pages suffered from cavernous empty space on wide viewports, and standard PATCH HTTP requests struggled with PHP multipart form parsing boundaries. We diagnosed and resolved this by restructuring the settings layout into a balanced `max-w-5xl` container using modular card components, configuring dual HTTP method routing (`Route::match(['patch', 'post'])`), and implementing automated Pest feature tests to guarantee atomic disk storage cleanup whenever avatars were replaced or deleted.

## 3. What would I plan differently next time?

In future agentic projects, I would establish design layout tokens and reusable UI card primitives during initial setup rather than refining them after backend feature implementation. Furthermore, I would embed automated headless browser visual regression checks directly into subagent milestone gates from day one, catching UX and spacing gaps before proceeding to release verification.